

Práctica: Navegación Inferior (BottomNavigationBar) en Flutter

Objetivo

Implementar un sistema de navegación por pestañas usando **BottomNavigationBar** en Flutter, comprendiendo la relación entre pantallas, navegación y estado.

1. Introducción teórica

La **barra de navegación inferior (BottomNavigationBar)** permite mostrar diferentes secciones o vistas principales dentro de una aplicación.

Cada pestaña puede representar una pantalla distinta o un componente con su propio contenido.

Es común en aplicaciones modernas como **Instagram**, **YouTube** o **Gmail**.

En Flutter, el **BottomNavigationBar** se combina con un **StatefulWidget** para controlar el índice de la pestaña seleccionada y mostrar el contenido correspondiente usando listas o estructuras condicionales.

Esto permite una navegación fluida entre vistas sin recargar la aplicación.

2. Ejemplo base

A continuación se muestra un ejemplo completo de una aplicación con tres pestañas: **Inicio**, **Usuarios** y **Configuración**.

Código:

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});  
  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

```

class _MyHomePageState extends State<MyHomePage> {
  int _selectedIndex = 0;

  final List<Widget> _pages = const [
    Center(child: Text('Inicio', style: TextStyle(fontSize: 24))),
    Center(child: Text('Usuarios', style: TextStyle(fontSize: 24))),
    Center(child: Text('Configuración', style: TextStyle(fontSize:
24))),
  ];

  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Ejemplo
BottomNavigationBar')),
      body: _pages[_selectedIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: _onItemTapped,
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label:
'Inicio'),
          BottomNavigationBarItem(icon: Icon(Icons.people), label:
'Usuarios'),
          BottomNavigationBarItem(icon: Icon(Icons.settings), label:
'Config'),
        ],
      ),
    );
  }
}

```

3. Actividades extra

Realiza las siguientes extensiones sobre el proyecto anterior para afianzar la comprensión de la navegación y el paso de datos entre pantallas.

1. Agrega un cuarto tab llamado **“Perfil”**.
2. Dentro del tab **“Perfil”**, muestra un botón que navegue a una pantalla nueva llamada **“Editar perfil”**.
3. En la pantalla **“Editar perfil”**, coloca un `TextField` para capturar el nombre del usuario.
4. Usa `Navigator.pop(context, nombre)` para regresar el valor capturado y mostrarlo en el tab **“Perfil”**.
5. Asegúrate de actualizar el estado para reflejar el nombre en pantalla.

Ejemplo de navegación para Editar Perfil

```
// Dentro del tab Perfil
ElevatedButton(
  onPressed: () async {
    final nombre = await Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const
EditarPerfilScreen()),
    );
    if (nombre != null) {
      setState(() {
        _nombreUsuario = nombre;
      });
    }
  },
  child: const Text('Editar perfil'),
);

// Pantalla Editar Perfil
```

```

class EditarPerfilScreen extends StatefulWidget {
  const EditarPerfilScreen({super.key});

  @override
  State<EditarPerfilScreen> createState() =>
    _EditarPerfilScreenState();
}

class _EditarPerfilScreenState extends State<EditarPerfilScreen> {
  final TextEditingController _controller = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Editar perfil')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(labelText: 'Nombre'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context, _controller.text);
              },
              child: const Text('Guardar'),
            ),
          ],
        ),
      ),
    );
  }
}

```

4. Preguntas de reflexión

1. ¿Por qué se recomienda usar un **StatefulWidget** para manejar el índice de la pestaña seleccionada?
 2. ¿Qué ventajas ofrece separar cada pestaña en su propio widget o pantalla?
 3. ¿Cómo cambia la navegación entre pantallas al usar **rutas nombradas** en lugar de `Navigator.push()`?
 4. ¿De qué forma el uso de `Navigator.pop(context, data)` facilita la comunicación entre pantallas?
 5. ¿Qué posibles mejoras o extensiones podrías agregar a esta aplicación para hacerla más completa?
-

Conclusión

Esta práctica refuerza el manejo del **estado**, la **navegación entre pantallas** y el **diseño modular** mediante pestañas.

Además, introduce la **comunicación entre pantallas** utilizando `Navigator.pop` con paso de datos, un concepto esencial en el desarrollo de aplicaciones Flutter.